H2S-NN: Heterogeneous Sheaf–Hodge Neural Networks with Adaptive Schema-to-Cell Selection

Peeradon Sarnkaew
peeradonsarnkaew46@gmail.com

Abstract

Heterogeneous graphs combine several node and relation types, while many of their observed relations are heterophilic. Conventional heterogeneous graph neural networks represent typed paths as message-passing channels, and existing heterogeneous sheaf networks learn a geometry on the observed graph but remain confined to its nodes and edges. We introduce H2S-NN, a heterogeneous sheaf–Hodge architecture in which concrete typed path instances provide higher-order, face-like supports for diffusion over an explicit edge cochain. Relation-conditioned node-to-edge maps define a learned first coboundary, an ordered sequence encoder constructs edge-to-face maps, and matrix-free lower and upper Hodge terms evolve node and edge states without materializing global block matrices. We further introduce AMPSS, a two-stage selection procedure that enumerates typed schemas, ranks them using training-only task signals and structural coverage, and refines the retained schemas and instances through differentiable nonnegative gates inside the upper edge Laplacian. We state precisely where the current construction differs from a strict cellular cochain complex: synthetic closure pairs correct the node Laplacian, whereas the learned second coboundary remains path-edge-only and $\delta_1\delta_0 = 0$ is encouraged rather than guaranteed. Current artifact-backed experiments establish end-to-end execution after the latest gate and HGB-protocol fixes, but the full repeated-seed baseline and ablation matrices remain in progress; accordingly, we make no state-of-the-art claim. The in-development THS-POOL graph pooling extension is intentionally outside the completed method.

1 Introduction

Graphs encountered in bibliographic networks, recommender systems, biology, and knowledge graphs rarely consist of a single kind of entity connected by a single kind of relation. A paper, an author, and a venue occupy different semantic roles; likewise, authorship, citation, and publication relations do not express the same compatibility. Treating such a network as homogeneous erases this distinction. Treating every observed edge as an instruction to average neighbouring features creates a second difficulty: heterogeneous relations are often heterophilic, so adjacent entities need not share a label or even inhabit comparable raw feature spaces.

Two established approaches address complementary parts of this problem. Heterogeneous GNNs use typed parameters, attention, or meta-paths to preserve semantic information [Wang et al., 2019, Fu et al., 2020, Hu et al., 2020]. Meta-path models are particularly attractive because a typed path such as paper–author–paper describes a relation that is not visible from either edge in isolation. Nevertheless, the path usually serves only as a route for collecting neighbours; after aggregation, the concrete path instance has no further mathematical role. Sheaf neural networks instead attach local vector spaces and learned restriction maps to a graph, replacing direct feature averaging with comparison after learned transport. Neural Sheaf Diffusion demonstrated the value of this geometry for heterophily

Preprint. Under review.

and oversmoothing [Bodnar et al., 2022], and HetSheaf specialized restriction-map prediction to heterogeneous graphs [Braithwaite et al., 2024]. These models, however, operate on the observed 1-skeleton.

H2S-NN connects the two perspectives by changing what a typed path becomes. Each selected concrete path instance is retained as a face-like higher-order support. Its ordered boundary edges are encoded into restriction maps, enter a learned second coboundary δ_1, and therefore modify the upper term $\delta_1^{\mathsf{T}}\delta_1$ of an edge Hodge operator. The model maintains both a node cochain and an independently initialized edge cochain. It diffuses the former with a learned L_0 and the latter with both lower and upper components of L_1. Thus the selected path affects the operator itself rather than merely adding another neighbourhood view.

Manual meta-paths are useful when reliable domain knowledge exists, but they are difficult to scale across schemas. We therefore develop AMPSS, an automatic candidate-pool construction and refinement procedure. A bounded type-graph search first proposes schemas. Training-only task signals, coverage, diversity, and redundancy determine a compact pool. Finally, a schema gate α_σ and an instance gate β_f weight upper-Laplacian summands during training. The combinatorial preselection is not differentiable, while refinement of the fixed pool is differentiable through the task and regularization objectives.

The contributions of this work are as follows.

1. We formulate typed path instances as operator-level higher-order supports for a heterogeneous sheaf model. Learned edge-to-face maps make the selected paths act through the upper edge Laplacian rather than through a semantic-channel sum.
2. We introduce a matrix-free heterogeneous sheaf–Hodge backbone that learns relation-conditioned node-to-edge maps, initializes an explicit edge cochain, and performs diffusion over node and edge ranks.
3. We propose AMPSS, a two-stage schema-to-cell selection mechanism with training-only preselection, hierarchical schema/instance gates, and partial-schema propagation.
4. We give a code-grounded account of the construction's guarantees and limitations, including positive semidefiniteness of the weighted Hodge terms, the absence of an exact chain condition, the task-dependent visibility of the edge cochain, and the empirical work required for a final claim.

Automatic meta-path generation alone is not the novelty: LMSPS and differentiable meta-graph or meta-multigraph search already automate schema selection [Li et al., 2024, 2022]. Our distinction is operator-level: selected instances become inputs to a learned second coboundary and thereby reshape edge diffusion.

2 Background and Related Work

2.1 Heterogeneous graphs and typed paths

A heterogeneous graph is a tuple $G = (V, E, \phi, \psi)$, where $\phi : V \to \mathcal{T}$ and $\psi : E \to \mathcal{R}$ assign node and relation types. A meta-path schema $\sigma = (t_0, r_1, t_1, \ldots, r_\ell, t_\ell)$ specifies a valid walk in the type graph. A concrete instance $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$ follows that schema in G. HAN and MAGNN aggregate features over manually specified meta-path views [Wang et al., 2019, Fu et al., 2020], whereas HGT uses typed attention without requiring explicit paths [Hu et al., 2020]. LMSPS progressively searches long-range paths, and PMMM searches a differentiable meta-multigraph while using partial message propagation to improve stability [Li et al., 2024, 2022]. These methods search or weight communication structures; they do not retain each path instance as an element of a second coboundary operator.

2.2 Cellular sheaves and neural diffusion

A cellular sheaf $\mathcal{F}$ on a graph assigns a vector space $\mathcal{F}(v)$ to each node, a vector space $\mathcal{F}(e)$ to each edge, and a linear restriction map $F_{v \trianglelefteq e} : \mathcal{F}(v) \to \mathcal{F}(e)$ for each incident pair. With a chosen edge orientation, the first coboundary compares transported endpoint signals,

$$(\delta_0 x)_e = F_{t(e) \trianglelefteq e} x_{t(e)} - F_{s(e) \trianglelefteq e} x_{s(e)}. \tag{1}$$

Table 1: Position of H2S-NN relative to adjacent model families.

Family	Primary mechanism	Difference in H2S-NN
HAN / MAGNN	Meta-path views route and aggregate messages.	Concrete path instances remain in an edge-to-face operator.
HGT	Typed attention over the observed graph.	Adds an explicit edge cochain and a learned upper edge term.
LMSPS / PMMM	Automatically select schemas or meta-multigraph channels.	Selected structures weight $\delta_1^{\mathsf{T}}\delta_1$, not only aggregation.
Neural Sheaf Diffusion	Learned sheaf diffusion on a homogeneous 1-skeleton.	Conditions on heterogeneous types and introduces path-supported edge diffusion.
HetSheaf	Heterogeneous restriction maps and gauge-aware readout.	Adds ordered edge-to-face maps and an explicit edge Hodge operator.

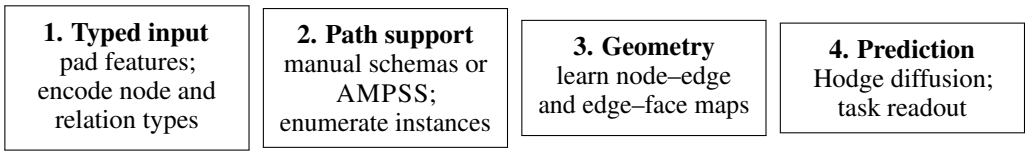


Figure 1: The H2S-NN pipeline. Selected path instances act through learned edge-to-face maps and the upper component of the edge operator.

The associated sheaf Laplacian is $L_0 = \delta_0^{\mathsf{T}}\delta_0$. When all stalks are one-dimensional and all restrictions are identities, this reduces to the ordinary graph Laplacian. Neural Sheaf Diffusion learns the restrictions from data and uses the induced geometry to control diffusion in heterophilic settings [Bodnar et al., 2022]. HetSheaf conditions the predictor on node and edge types and develops a gauge-aware graph readout [Braithwaite et al., 2024]. H2S-NN retains typed restriction learning while adding an explicit edge cochain and a path-supported second operator.

2.3 Heterophily-aware heterogeneous learning

Recent methods address heterogeneous heterophily by rewiring, spectral filtering, latent-graph construction, contrastive objectives, or cross-type homophily measurements [Guo et al., 2023, Lu et al., 2024, Shen and Kang, 2024, Wang et al., 2025, Tao et al., 2025]. These approaches are important baselines because they target the same failure mode of naive neighbour averaging. Their mechanism is nevertheless different from learning maps on node–edge and edge–face incidences.

2.4 Positioning

Table 1 summarizes the boundary of the contribution. We use "face-like" deliberately: Section 3.6 explains why the implemented operators form a generalized path-supported cochain system rather than an exact cellular cochain complex.

3 H2S-NN

3.1 Problem setting and notation

Let $N = |V|$ and $M = |E|$. We use a common stalk dimension d for node, edge, and face-output vectors, a relation-embedding dimension d_r, and a sequence hidden dimension d_h. Node and edge cochains are $x \in \mathbb{R}^{N\times d}$ and $s \in \mathbb{R}^{M\times d}$. The F selected path instances contain $I = \sum_{f=1}^{F} |\partial f|$ path-edge incidences. Superscripts "src" and "tgt" denote the two node-to-edge restriction maps of an oriented edge; $A_{e\triangleright f}$ denotes an edge-to-face map. Adjoint operators are written with T because the implementation uses the Euclidean inner product on every stalk.

3.2 Typed input representation

Raw feature matrices may have different widths across node types. The data layer pads them to a common width and appends relation-level structural statistics derived using training labels only. For a node v of type $\phi(v)$, the implemented encoder is

$$h_v^{(0)} = \mathrm{Dropout}\big(\mathrm{GELU}\big(\mathrm{LN}\big(W_{\phi(v)}x_v + b_{\phi(v)}\big) + q_{\phi(v)}\big)\big) \in \mathbb{R}^d, \tag{2}$$

where each node type has a separate affine map and q_t is a learned type embedding. If a graph has no node features, a free embedding table supplies one learned d-vector per node; this avoids the bias-only collapse of a zero-width linear layer. Each relation r has an embedding $z_r \in \mathbb{R}^{d_r}$.

3.3 Relation-conditioned node-to-edge maps

For every stored directed edge $e = (u \xrightarrow{r} v)$, two relation-specific MLPs receive $[h_u^{(0)} \,\|\, h_v^{(0)} \,\|\, z_r]$ and predict maps $F_e^{\mathrm{src}}, F_e^{\mathrm{tgt}} \in \mathbb{R}^{d\times d}$. The default maps are conformal,

$$F_e = \lambda_e Q_e, \qquad Q_e^{\mathsf{T}}Q_e = I, qquad\lambda_e = \exp(\min\{a_e, 2\}), \tag{3}$$

with Q_e produced by a pure-PyTorch Householder parameterization. Cayley maps are also implemented, while Euler rotations are restricted to $d \in \{2, 3\}$. A matrix-exponential option is rejected during configuration validation because its full-edge CUDA backward path was unsafe in the target environment. We call the maps *scaled orthogonal*, not orthogonal.

The learned first coboundary is

$$(\delta_0 h^{(0)})_e = F_e^{\mathrm{tgt}} h_v^{(0)} - F_e^{\mathrm{src}} h_u^{(0)}. \tag{4}$$

It is used as an edge feature, but not as the initial edge signal itself.

3.4 Independent edge-cochain initialization

The edge initializer preserves information that would be lost by forcing every initial edge state to be exact:

$$s_e^{(0)} = \mathrm{MLP}_{\mathrm{init}}\Big(h_u^{(0)} \,\|\, h_v^{(0)} \,\|\, z_r \,\|\, (\delta_0 h^{(0)})_e\Big). \tag{5}$$

The final block is optional through `use_delta0_init`. Initializing $s^{(0)} = \delta_0 h^{(0)}$ would restrict the edge signal to the image of δ_0 and remove an independent component before diffusion begins.

3.5 Typed path instances and edge-to-face maps

In manual mode, dataset-specific templates determine which paths are enumerated. In automatic mode, Section 4 constructs a fixed candidate pool. For a concrete path instance $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$, the sequence module forms tokens

$$g_i = \mathrm{MLP}_g\Big(h_{v_{i-1}}^{(0)} \,\|\, h_{v_i}^{(0)} \,\|\, z_{\psi(e_i)}\Big), \qquad (o_1, \ldots, o_\ell) = \mathrm{LSTM}(g_1, \ldots, g_\ell). \tag{6}$$

A learned vector a gives position attention

$$\pi_{fi} = \mathrm{softmax}_i\left(\frac{o_i^{\mathsf{T}} a}{\sqrt{d_h}}\right), \qquad A_{e_i \triangleright f} = \pi_{fi}\,\mathrm{reshape}(W_A o_i + b_A, d, d). \tag{7}$$

The attention weight multiplies the map itself. Maps are produced in bounded face chunks, cached once per forward pass, and reused for all diffusion steps.

3.6 Closure convention and the chain-condition caveat

Every open path has an endpoint pair (v_ℓ, v_0) recorded as a synthetic closure edge. The implementation does not add a stateful closure edge to s and does not include that edge in δ_1. Instead, it adds the closed-form contribution of a frozen γI_d restriction to the node Laplacian,

$$C_{\mathrm{cl}}(x)_{v_\ell} \mathrel{+}= \gamma^2(x_{v_\ell} - x_{v_0}), \qquad C_{\mathrm{cl}}(x)_{v_0} \mathrel{-}= \gamma^2(x_{v_\ell} - x_{v_0}), \tag{8}$$

for every recorded closure occurrence. The default $\gamma = 10^{-6}$ yields an effective 10^{-12} coupling.

Because the second operator sees only the stored path edges, independently learned restrictions do not guarantee $\delta_1\delta_0 = 0$. We therefore describe the implementation as a generalized, path-supported sheaf–Hodge system and penalize the composition during training. A future strict cellular version would materialize a compatible closed boundary and enforce the chain condition by construction.

3.7 Matrix-free coboundaries and Hodge operators

Let $\eta(e, f) \in \{-1, +1\}$ be the stored incidence sign. The second operator is

$$(\delta_1 s)_f = \sum_{e \in \partial_{\text{path}} f} \eta(e, f) A_{e \triangleright f} s_e. \tag{9}$$

The code supports arbitrary signs and tests a negative-incidence case. Production enumeration follows stored forward relations, so it currently emits $+1$ for every incidence. The node and edge operators are

$$L_0 x = \delta_0^{\mathsf{T}} \delta_0 x + C_{\text{cl}}(x), \tag{10}$$

$$L_1 s = \underbrace{\delta_0 \delta_0^{\mathsf{T}} s}_{L_1^{\text{down}} s} + \underbrace{\delta_1^{\mathsf{T}} \delta_1 s}_{L_1^{\text{up}} s}. \tag{11}$$

No global $Nd \times Nd$ or $Md \times Md$ matrix is formed. Batched matrix–vector products and scatter-add operations implement each operator and its adjoint.

3.8 Hodge diffusion

The model performs K explicit Euler steps,

$$x^{(k+1)} = x^{(k)} - \eta_0 L_0 x^{(k)}, \qquad s^{(k+1)} = s^{(k)} - \eta_1 L_1 s^{(k)}, \quad k = 0, \ldots, K-1. \tag{12}$$

Learnable step sizes are softplus-parameterized and therefore positive. Optional layer dropout is applied to both ranks after each step. Layer normalization produces $x^\star$ and $s^\star$.

The ranks are evolved in parallel under operators inferred from the same initial node state; the implemented recurrence does not feed $s^{(k)}$ into the node update or $x^{(k)}$ into the edge update. Higher-order information therefore reaches a task only if its readout consumes $s^\star$ or a descriptor derived from it. This observation is central to Section 6.

3.9 Task readouts

Node classification. Each node receives the mean of its incident edge states, $\bar{s}_v = |E(v)|^{-1} \sum_{e \ni v} s_e^\star$. Target-type logits are

$$\widehat{y}_v = \text{MLP}_{\text{node}}(x_v^\star \,\|\, h_v^{(0)} \,\|\, \bar{s}_v). \tag{13}$$

This is the present readout through which the face pathway directly affects supervised predictions.

Link prediction. The implemented dot, DistMult, and concatenation-MLP decoders score pairs using $x^\star$. DistMult uses one relation matrix per pooled target relation. Since these decoders do not consume $s^\star$, the current link-prediction task loss does not directly supervise the upper edge operator; link experiments validate the shared encoder and data protocol, not the central higher-order readout claim.

Graph classification. The default checked-in configurations use mean pooling of $x^\star$. Experimental terminal heads that also inspect edge descriptors exist, but the planned hierarchical THS-POOL method is not complete. Graph classification is therefore implementation coverage and future evaluation rather than evidence for higher-order pooling in this paper.

3.10 Training objective

For single-label tasks, $\mathcal{L}_{\text{task}}$ is cross-entropy; IMDB uses multi-label binary cross-entropy with logits. Link prediction uses binary cross-entropy. Four geometric terms are shared across trainers:

$$\mathcal{L}_0 = \lambda_0 \,\text{mean}\, \|\delta_0 x^\star\|_2^2, \qquad \mathcal{L}_1 = \lambda_1 \,\text{mean}\, \|\delta_1 s^\star\|_2^2, \tag{14}$$

$$\mathcal{L}_{\text{cons}} = \lambda_c \,\text{mean}\, \|s^\star - \delta_0 x^\star\|_2^2, \qquad \mathcal{L}_{\text{chain}} = \lambda_{co} \,\text{mean}\, \left\|\delta_1 \delta_0 h^{(0)}\right\|_2^2. \tag{15}$$

The last term is a soft chain-consistency penalty, not a proof that the learned maps form an exact complex. The complete loss is

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \mathcal{L}_0 + \mathcal{L}_1 + \mathcal{L}_{\text{cons}} + \mathcal{L}_{\text{chain}} + \rho(t)\mathcal{L}_{\text{disc}}, \tag{16}$$

where $\rho(t)$ is zero during discovery warm-up and optionally increases linearly afterward.

4 Adaptive Schema-to-Cell Selection

4.1 Candidate generation and statistics

AMPSS begins with a directed type graph whose arcs are the stored relation tuples. Bounded depth-first search enumerates schemas of length $\ell \in [\ell_{\min}, \ell_{\max}]$, optionally requiring the target type at both ends and allowing repeated types. Candidate count, instances per anchor, instances per schema, and total materialized instances are separately capped. This is a preprocessing operation and is non-differentiable.

For node classification, labels on the training mask are propagated for a fixed number of steps to give distributions $\widehat{y}_v$. A sampled instance receives

$$d(f) = \|\widehat{y}_{v_0} - \widehat{y}_{v_\ell}\|_2^2, qquad D(\sigma) = \frac{1}{|\mathcal{I}_\sigma|} \sum_{f \in \mathcal{I}_\sigma} d(f). \tag{17}$$

Only training labels contribute to $\widehat{y}$. Link prediction uses an analogous two-channel role signal constructed from training-positive source and target endpoints. Validation and test positives are excluded. For graph classification, the pool is learned from training graphs only: schema presence is counted by class and scored by the variance of class-normalized occurrence, with a small log-frequency tie breaker. This graph-level score is task-specific and should not be conflated with Equation 17.

The node/link path also records instance count, start and end coverage, relation diversity, type diversity, average branching, estimated cost, and sampled endpoint sets. The structural instance feature tensor is stored for analysis but is not currently an input to β_f.

4.2 Greedy pool and differentiable refinement

For node and link tasks, if S is the set already selected, the score is

$$q(\sigma \mid S) = \frac{D(\sigma)^{a_d}\, C(\sigma)^{a_c}\, V(\sigma)^{a_v}}{1 + \max_{\sigma' \in S} J(\sigma, \sigma')}, \tag{18}$$

where C is end coverage, V is type diversity, and J is the Jaccard overlap of sampled endpoint sets. The implementation uses $(a_d, a_c, a_v) = (1, 0.5, 0.3)$. After selecting k schemas, scores are recomputed leave-one-out for gate initialization.

The selected pool receives one gate per schema and one state-dependent gate per face:

$$\alpha_\sigma = \text{sigmoid}(\theta_\sigma), \tag{19}$$

$$\beta_f = \text{sigmoid}\Big(\text{MLP}_\beta(h_{v_0}^{(0)} \parallel h_{v_\ell}^{(0)} \parallel e_\sigma)\Big), \qquad w_f = \alpha_{\sigma(f)}\beta_f. \tag{20}$$

The logits θ are initialized from standardized log topology scores, scaled by a temperature and clamped. Directly setting $\theta = \log q$ placed every gate in the near-zero saturated region in an earlier implementation.

Algorithm 1 AMPSS preprocessing and refinement

Require: Relation tuples, training split, candidate and instance budgets
Build the directed type graph and enumerate bounded typed schemas
Sample instances and compute task-specific training-only statistics
Greedily retain a compact, non-redundant schema pool
Materialize path groups, incidence signs, and closure endpoint pairs
Initialize α logits from standardized log topology scores
for each training epoch **do**
    Use uniform faces during warm-up; otherwise compute α and β
    Apply partial-schema propagation and set $w_f = \alpha_{\sigma(f)}\beta_f$
    Run matrix-free Hodge diffusion and the task readout
    Optimize Equation 16
end for

During training, partial-schema propagation independently retains each schema with probability p and rescales its gate by $1/p$. This adapts the stabilization principle of partial message propagation [Li et al., 2022]. The weighted operator scales each face row of δ_1 by $\sqrt{w_f}$:

$$L_1^{\mathrm{up}} s = \sum_{\sigma} \sum_{f \in \mathcal{I}_\sigma} w_f B_f^{\mathsf{T}} B_f s, \qquad B_f s = \sum_{e \in \partial_{\mathrm{path}} f} \eta(e, f) A_{e \triangleright f} s_e. \tag{21}$$

The discovery regularizer is

$$\begin{aligned} \mathcal{L}_{\mathrm{disc}} = \lambda_s \mathcal{L}_{\text{sparse-schema}} + \lambda_i \mathcal{L}_{\text{sparse-instance}} + \lambda_r \mathcal{L}_{\mathrm{redundancy}} \\ + \lambda_v \mathcal{L}_{\mathrm{coverage}} + \lambda_{\mathrm{cost}} \mathcal{L}_{\mathrm{cost}} + \lambda_H \mathcal{L}_{\mathrm{entropy}}. \end{aligned} \tag{22}$$

Schema and instance sparsity minimize mean gate values; coverage is a negative reward; cost penalizes long, frequent schemas; and the positive entropy coefficient sharpens schema gates. The inexpensive redundancy term uses a coverage profile rather than an exact pairwise instance-overlap matrix. Instance sparsity consumes a gradient-attached gate mean; a detached copy is logging-only.

5 Operator Properties and Computational Cost

Proposition 1 (Positive semidefiniteness)**.** *For fixed restriction maps and nonnegative face weights, L_0 and L_1 in Equations 10–21 are self-adjoint positive semidefinite.*

Proof. For every node cochain x, $\langle x, L_0 x\rangle = \|\delta_0 x\|_2^2 + \sum_c \gamma^2 \left\|x_{t(c)} - x_{s(c)}\right\|_2^2 \geq 0$. For every edge cochain s, $\langle s, L_1 s\rangle = \left\|\delta_0^{\mathsf{T}} s\right\|_2^2 + \sum_f w_f \left\|B_f s\right\|_2^2 \geq 0$. Each expression is a sum of Gram terms. □

The sigmoid parameterization makes $w_f \geq 0$. Partial-schema propagation preserves nonnegativity, even when rescaling makes a retained weight larger than one.

Proposition 2 (One-step energy stability)**.** *For a fixed positive-semidefinite operator L, $z^+ = z - \eta L z$ is non-expansive in Euclidean norm when $0 \leq \eta \leq 2/\lambda_{\max}(L)$.*

Proof. Diagonalize $L = U \Lambda U^{\mathsf{T}}$. Each spectral component is multiplied by $1 - \eta\lambda_i$, whose magnitude is at most one under the stated condition. □

The implementation enforces $\eta > 0$ but not the spectral upper bound. The proposition is a sufficient condition, not an automatic guarantee of every trained configuration. Layer normalization and gradient clipping are practical safeguards.

Permutation behaviour. Predictors are shared by type or relation, and adjoints use indexed scatter-add. A consistent relabeling of nodes, edges, and path instances therefore permutes intermediate cochains equivariantly. Target selection and symmetric pooling remove the final indexing dependence.

Table 2: Implemented datasets and their present role in the evaluation.

Task	Datasets	Source	Protocol status
Node classification	ACM, DBLP, IMDB	PyG HGBDataset	Primary; HGB masks, with validation carved from training when absent.
Link prediction	LastFM, PubMed, Amazon, YouTube	Local HGB files	Official targets and fixed test labels where shipped; auxiliary until edge-aware readout.
Link prediction	AmazonBook	PyG AmazonBook	Deterministic positive split; never conflated with HGB Amazon.
Graph classification	MUTAG, PROTEINS, ENZYMES, NCI1	PyG TUDataset	Stratified 80/10/10 hold-out; auxiliary until pooling is completed.

Cost. Once dense $d \times d$ restrictions are available, one L_0 application costs $O(Md^2)$ and one L_1 application costs $O((M+I)d^2)$. A K-step rollout costs $O(K(M+I)d^2)$ for operator application. Dense map storage is $O(Md^2 + Id^2)$, and Householder/Cayley construction can dominate at larger d. The code chunks face maps and rejects CUDA configurations whose estimated node-to-edge materialization exceeds a configurable fraction (default 0.20) of device memory.

6 Experimental Evaluation

The experimental design follows the breadth of the HetSheaf evaluation [Braithwaite et al., 2024], but distinguishes a completed implementation path from a completed claim. The central higher-order claim is evaluated through node classification because the node readout consumes the edge cochain. Link and graph tasks remain auxiliary until their readouts expose the same higher-order signal.

6.1 Research questions

RQ1. Does path-supported edge diffusion improve target-node classification over graph-only sheaf diffusion and strong heterogeneous baselines?

RQ2. Are gains attributable to the upper edge term, learned restrictions, and independent edge initialization rather than parameter count alone?

RQ3. Does AMPSS match or improve manual schemas, and are learned α/β stable across seeds?

RQ4. What accuracy, memory, and runtime trade-offs result from stalk dimension, face budget, path length, and map parameterization?

6.2 Datasets and protocols

For HGB node classification, all graph relations remain available in the transductive graph. Schema scoring sees only the final training mask. IMDB retains multi-hot genre labels and uses sigmoid predictions with macro- and micro-F1. For local-HGB link prediction, validation positives are carved from the provided training graph while preserving at least one training edge per source where possible. Validation negatives are uniformly sampled, and official fixed test triples are used when supplied. Held-out target-relation edges are removed from the message-passing graph; AmazonBook also removes reverse-relation copies. The loaders contain explicit overlap assertions.

Table 3: Interim artifact-backed results (mean $\pm$ sample standard deviation, percent).

Dataset	Configuration	Runs	Observed metrics	Artifact scope
ACM	H2S-NN + AMPSS	2/5	Macro-F1 85.83 ± 0.95; Micro-F1 85.88 ± 0.87	Post-fix tuning runs
LastFM	H2S-NN link encoder	3/5	AUPR 64.38 ± 1.57; AUROC 63.17 ± 0.97; MRR 85.11 ± 1.88	Post-fix tuning runs

Table 4: Evidence required before this working manuscript becomes a submission claim.

Study	Required output	Current status
Node classification	Five or more fixed-protocol seeds on ACM/DBLP/IMDB, plus paired tests.	Two post-fix ACM runs; DBLP/IMDB pending.
Structural ablations	No faces, no δ_1, identity maps, uniform gates, manual, α only, full $\alpha\beta$.	First four switches exist; full matrix pending.
Efficiency	Runtime, peak memory, parameters, schema/face counts, and path-incidence count.	Instrumentation exists; controlled comparison pending.
Link prediction	Readout exposing $s^\star$, or explicit encoder-only scope.	Current decoder is node-state-only.
Graph classification	Valid higher-order readout followed by repeated-seed evaluation.	THS-POOL incomplete.

6.3 Baselines, ablations, and model selection

The minimum node comparison includes GCN/GAT [Kipf and Welling, 2017, Veličković et al., 2018], HAN/HGT [Wang et al., 2019, Hu et al., 2020], Neural Sheaf Diffusion [Bodnar et al., 2022], HetSheaf [Braithwaite et al., 2024], and heterophily-aware methods [Guo et al., 2023, Lu et al., 2024, Shen and Kang, 2024]. Automatic-selection comparisons include LMSPS and PMMM [Li et al., 2024, 2022]. Published numbers are not mixed with new runs unless dataset identity, split, and metric definitions match.

Implemented ablations are no faces, no δ_1, identity node-to-edge maps, and uniform gates. The final matrix additionally requires manual versus automatic schemas, α-only versus $\alpha\beta$, equal-parameter controls, closure sensitivity, and path/face-budget sweeps.

All experiments use validation-selected checkpoints, Adam or AdamW, gradient clipping, and cosine or reduce-on-plateau scheduling. W&B tuning executes each trial in a fresh worker, fingerprints resume state, and rejects stale search spaces. Every run saves its resolved configuration, log, JSONL metrics, last checkpoint, and best checkpoint.

7 Current Results and Evidence Status

Table 3 contains only summaries reconstructible from current run artifacts after the schema-gate initialization and HGB loader fixes. Each run independently selects a configuration by validation performance; the table is incomplete relative to the planned five-run protocol. It is included to define the evidence boundary, not as a final benchmark table.

These measurements do not support the earlier draft's claim that H2S-NN outperforms all baselines. They establish narrower facts: corrected discovery gates are active on ACM; the official-HGB LastFM path executes across several seeds; and current performance leaves substantial room for architectural and optimization work. Earlier DBLP, IMDB, graph-classification, clustering, efficiency, and four-way ablation headline tables are excluded because matching post-fix artifacts are unavailable.

8 Discussion

The principal conceptual step is the promotion of selected path instances from routing rules to terms in a learned edge operator. Node-to-edge maps allow relation-specific comparison of heterogeneous

endpoints; edge-to-face maps preserve the ordered typed boundary; and nonnegative gates select which higher-order supports shape diffusion. Because node and edge states diffuse independently, the impact of a face is localized to the edge rank and rank-aware readouts. This makes no-δ_1 and readout ablations especially informative, while preventing us from crediting faces for logits that never observe $s^\star$.

The current limitations are substantive. Closure pairs affect L_0 but are absent from δ_1, so the chain condition is soft rather than structural. Candidate preselection is automatic but non-differentiable. Signed incidence is implemented, yet production DFS emits only positive signs. Every active incidence may carry a dense $d \times d$ map. The repeated-seed baseline and ablation suite is unfinished. Finally, THS-POOL (internally S^2Pool-H2) remains in development. The older terminal projector named S2Pool is a separate experimental head and can degenerate when its descriptor stack is full column rank; neither is claimed as a completed contribution.

Immediate future work is a strict closed-boundary construction with a structural chain condition, lower-rank map parameterizations, and coupled node–edge diffusion. On graph tasks, THS-POOL should be evaluated only after its rank-aware readout beats a node-only mean baseline under repeated seeds. Calibrated hard-concrete gates and uncertainty-aware schema stability measures may provide a more discrete, interpretable selected complex.

9 Conclusion

We presented H2S-NN, a heterogeneous sheaf–Hodge architecture that retains selected typed path instances as higher-order supports for an explicit edge diffusion operator. Learned relation-conditioned node-to-edge maps, sequence-conditioned edge-to-face maps, and matrix-free Hodge terms define the backbone. AMPSS complements it with training-only schema preselection and differentiable schema/instance refinement inside the upper edge Laplacian. The design gives typed paths an operator-level role absent from conventional meta-path aggregation. A code-grounded analysis also identifies the present boundary of the claim: the construction is not yet an exact cellular complex, higher-order information is visible only to rank-aware readouts, and the full empirical matrix is unfinished. These limitations define a concrete path from the current research prototype to a defensible final evaluation.

References

Cristian Bodnar, Francesco Di Giovanni, Benjamin P. Chamberlain, Pietro Liò, and Michael Bronstein. Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in GNNs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Luke Braithwaite, Alessio Borgi, Gabriele Onorato, Kristjan Tarantelli, Iulia Duta, Francesco Restuccia, Fabrizio Silvestri, and Pietro Liò. Heterogeneous sheaf neural networks. *arXiv preprint arXiv:2409.08036*, 2024.

Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of The Web Conference (WWW)*, 2020.

Jiayan Guo, Lun Du, Wendong Bi, Qiang Fu, Xiaojun Ma, Xu Chen, Shi Han, Dongmei Zhang, and Yan Zhang. Homophily-oriented heterogeneous graph rewiring. In *Proceedings of the ACM Web Conference (WWW)*, 2023.

Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference (WWW)*, 2020.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Chao Li, Hao Xu, and Kun He. Differentiable meta multigraph search with partial message propagation on heterogeneous information networks. *arXiv preprint arXiv:2211.14752*, 2022.

Chao Li, Zijie Guo, Qiuting He, Hao Xu, and Kun He. Long-range meta-path search on large-scale heterogeneous graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Kangkang Lu, Yanhua Yu, Zhiyong Huang, Yunshan Ma, Xiao Wang, Meiyu Liang, Yuling Wang, Yimeng Ren, and Tat-Seng Chua. Addressing graph heterogeneity and heterophily from a spectral perspective. *arXiv preprint arXiv:2410.13373*, 2024.

Zhixiang Shen and Zhao Kang. When heterophily meets heterogeneous graphs: Latent graphs guided unsupervised representation learning. *arXiv preprint arXiv:2409.00687*, 2024.

Zhen Tao, Ziyue Qiao, Chaoqi Chen, Zhengyi Yang, Lun Du, and Qingqiang Sun. On the cross-type homophily of heterogeneous graphs: Understanding and unleashing. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*, 2025.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Haosen Wang, Chenglong Shi, Can Xu, Surong Yan, and Pan Tang. Homophily-aware heterogeneous graph contrastive learning. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2025.

Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S. Yu. Heterogeneous graph attention network. In *The World Wide Web Conference (WWW)*, 2019.

A Notation and Full Forward Pass

Table 5: Main symbols used in the paper.

Symbol	Meaning
$G = (V, E, \phi, \psi)$	Directed heterogeneous graph with node- and relation-type maps.
N, M, F, I	Numbers of nodes, stored edges, path instances, and path-edge incidences.
d, d_r, d_h	Stalk, relation-embedding, and sequence hidden dimensions.
$h^{(0)}, x^\star, s^{(0)}, s^\star$	Initial/final node and edge cochains.
$F_e^{\mathrm{src}}, F_e^{\mathrm{tgt}}$	Learned node-to-edge restriction maps.
$A_{e \triangleright f}$	Learned edge-to-face restriction map.
δ_0, δ_1	First and path-supported second coboundary operators.
L_0, L_1	Node and edge Hodge-style Laplacians.
$\alpha_\sigma, \beta_f, w_f$	Schema, instance, and effective face weights.
γ	Frozen synthetic-closure restriction scale.

Algorithm 2 H2S-NN forward pass for node classification

Require: Typed graph tensors, face groups, closure pairs, target indices, epoch
1: Encode typed nodes using Equation 2
2: Predict $F^{\mathrm{src}}, F^{\mathrm{tgt}}$ and compute $\delta_0 h^{(0)}$
3: Initialize $s^{(0)}$ with Equation 5
4: Encode ordered path boundaries and cache $A_{e \triangleright f}$
5: Build uniform weights, or $\alpha\beta$ weights after discovery warm-up
6: **for** $k = 0$ to $K - 1$ **do**
7: Apply L_0, including the closure correction
8: Apply $L_1^{\mathrm{down}} + L_1^{\mathrm{up}}$
9: Take the Euler step and apply layer dropout
10: **end for**
11: Layer-normalize $x^\star, s^\star$ and compute target logits
12: Return logits, cochain diagnostics, and gate summaries

B Dataset and Task Details

LastFM targets $(\mathrm{user}, \mathrm{rel}_0, \mathrm{artist})$; PubMed targets $(\mathrm{disease}, \mathrm{rel}_2, \mathrm{disease})$; YouTube targets one user–user relation at a time; and HGB Amazon pools its two product–product relations while passing

Table 6: Checked-in manual node-classification schema presets.

Dataset	Target type	Manual schemas	Labels
ACM	paper	PAP, PSP	single-label, 3 classes
DBLP	author	APA, APTPA	single-label, 4 classes
IMDB	movie	MAM, MDM	multi-label, 5 genres

relation IDs to DistMult. AmazonBook is a separate user–book dataset and is never compared under the name "Amazon" to an HGB Amazon baseline.

For graph discovery, training-graph paths are grouped by the alternating typed signature. Candidate presence is counted at most once per graph for class association, while total instance count is retained as a cost/frequency feature. Selected training-derived names filter every mini-batch. No validation or test graph label enters template construction.

C Hyperparameters and Reproducibility

Table 7: Base values in checked-in ACM/DBLP/IMDB configurations. Tuning overrides are stored in each resolved run configuration.

Group	Field	Base value
Stalks	d, d_r, d_h	$32, 16, 64$
Diffusion	K, η_0, η_1	$4, 0.05, 0.05$
Maps	family	scaled Householder
Networks	edge/init/face/classifier hidden	128 each
Geometry	$\lambda_0, \lambda_1, \lambda_c, \lambda_{co}$	$0, 10^{-3}, 10^{-2}, 10^{-2}$
Optimizer	name, learning rate, weight decay	AdamW, 10^{-3}, 10^{-4}
Training	epochs, gradient clip, base seed	$200, 1.0, 42$
Discovery	schema length, selected k	2–4, dataset dependent (6 or 8)
Discovery	partial probability, warm-up	0.8, 5 epochs in AMPSS configs

The driver saves `config.yaml`, `run.log`, `metrics.jsonl`, `checkpoints/last.pt`, and `checkpoints/best.pt`. Validation metrics alone select the best epoch. Tuning state is fingerprinted per seed and search space. The test suite uses `unittest`; on the target Windows machine, PyG imports can be blocked by Application Control at `xxhash`, an environment-policy failure rather than a model failure.

D Required Final Ablation Matrix

Table 8: Minimum ablations and the question answered by each.

Ablation	Interpretation
No faces	Reduces to node/edge sheaf diffusion without path-supported upper terms.
No δ_1	Keeps data construction while deleting L_1^{up}.
Identity node-to-edge maps	Tests whether learned heterogeneous transport is necessary.
Uniform α, β	Separates discovery weights from face support.
Manual schemas	Compares automatic construction to expert templates.
α only	Separates schema-level from instance-level refinement; needs a dedicated control.
Equal-parameter control	Tests topology rather than additional capacity.
Closure sweep	Measures whether the very small default γ has a detectable effect.
Face/path budget sweep	Reports the accuracy–memory trade-off as F and I change.

Table 9: Source files grounding the methodological statements.

Component	Authoritative implementation
Typed encoding	`src/models/node_encoder.py`
Node–edge and edge–face maps	`src/models/sheaf_maps.py`
Operators and closure correction	`src/models/coboundary_ops.py`
Euler diffusion	`src/models/diffusion.py`
Schema candidates and scoring	`src/data/schema_discovery.py`
Discovery gates and losses	`src/models/schema_gates.py`; `src/models/discovery_losses.py`
Task readouts	`src/models/readout.py`, `link_readout.py`, `graph_pooling.py`
End-to-end wiring	`src/models/h2snn_model.py`

E Implementation Traceability

F Status of THS-Pool

THS-POOL is the working name used here for the hierarchical, typed, higher-order pooling project documented internally as S^2Pool-H2. Its intended design performs type-aware node selection, constructs a coarse typed graph, re-enumerates path supports at the coarse resolution, re-encodes the coarse stalks, and reads node, edge, and face ranks across resolutions. Only the `encode_from_stalk` backbone entry point and terminal diagnostic heads are currently in the codebase; the pooler, coarse driver, and final controlled evaluation are not complete. This appendix reserves the connection without treating THS-POOL as part of the present method or results.